

Capitolo 4

Programmazione Funzionale: Il Linguaggio Haskell

4.1 Dal Lambda Calcolo alla Programmazione Funzionale

- La programmazione funzionale fa parte del paradigma dichiarativo, radicalmente diverso dal paradigma imperativo originariamente sviluppato come mezzo per determinare il comportamento delle macchine programmabili riflettendone la struttura. In quest'ultimo paradigma il contenuto della memoria – divenuto fondamentale a seguito dell'idea di computer a programma memorizzato di Von Neumann – costituisce implicitamente lo stato della computazione, che quindi può essere modificato attraverso specifiche istruzioni di assegnamento alle variabili, da eseguire in una certa sequenza.
- In programmazione funzionale la computazione ha invece luogo attraverso la valutazione di espressioni basate su funzioni. I programmi risultanti sono in genere più concisi dei corrispondenti programmi imperativi e, soprattutto, sono rappresentati a un livello di astrazione più elevato cosicché sono più leggibili e meglio si prestano a essere analizzati con metodi formali. I valori delle variabili non cambiano più una volta che sono stati attribuiti (quindi non è possibile modificare ripetutamente il valore di una variabile) e le valutazioni di espressioni basate su funzioni non possono dar luogo a effetti collaterali al di fuori delle espressioni stesse (come la modifica di variabili globali o parametri passati per indirizzo).
- La programmazione funzionale raggiunge pertanto la trasparenza referenziale che sta alla base del ragionamento equazionale: eguali possono essere sostituiti a eguali. Poiché viene restituito sempre lo stesso risultato ogni volta che un'espressione basata su funzioni viene valutata sugli stessi argomenti, il valore di un'espressione complessa calcolato su determinati argomenti può essere memorizzato per usi successivi in modo da non doverlo ricalcolare.
- Il λ -calcolo è il prototipo di linguaggio di programmazione funzionale, cioè di linguaggio nel quale le funzioni sono entità di prima classe dotate di appositi tipi e possono essere ricorsive (cioè richiamare se stesse), polimorfe (cioè essere applicate ad argomenti di tipi diversi e/o restituire risultati di tipi diversi) e di ordine superiore (cioè avere funzioni come argomenti e/o restituire funzioni come risultati).
- Il primo linguaggio di programmazione funzionale è stato Lisp (List Processor), sviluppato negli anni 1950 da John McCarthy come linguaggio algebrico per la manipolazione di liste da usare per elaborazioni simboliche in intelligenza artificiale. Tra le sue caratteristiche principali citiamo l'uso di espressioni condizionali per descrivere agevolmente funzioni ricorsive, l'impiego di funzioni di ordine superiore sulle liste e la gestione dinamica delle liste completamente automatica e trasparente al programmatore sia in fase di allocazione che di deallocazione (garbage collection), oltre alla possibilità di rappresentare sia dati che programmi come espressioni. Il λ -calcolo ebbe un impatto minimale sul Lisp, che comprende anche costrutti imperativi come le istruzioni di assegnamento. Una maggiore purezza si trova nel suo principale dialetto, Scheme, sviluppato negli anni 1970 da Steele e Sussman. Esso ammette chiamate ricorsive solo in coda alla definizione delle funzioni (per motivi di efficienza) e la visibilità delle variabili solo all'interno delle funzioni in cui sono dichiarate (e non anche in quelle invocate da tali funzioni).

- Negli anni 1960 Peter Landin, dopo aver formalizzato la semantica di Algol nel λ -calcolo, sviluppò il linguaggio Iswim (If you See What I Mean) ispirandosi al λ -calcolo esteso con costanti. Sul piano sintattico, Iswim abbandona la notazione prefissa in favore di quella infissa, evita l'uso pesante delle parentesi tonde di Lisp e introduce altresì i costrutti **let-in** e **where**, per dichiarare identificatori all'interno di definizioni di funzioni, come pure l'uso dell'indentazione, al posto di terminatori o separatori quali virgola e punto e virgola. Sul piano semantico, Iswim si fonda sull'idea di costruire linguaggi sintatticamente ricchi sulla base di linguaggi sintatticamente parsimoniosi ma sufficientemente espressivi, sul ragionamento equazionale e sul concetto di macchina astratta su cui eseguire i programmi.
- Una spinta fondamentale alla diffusione della programmazione funzionale venne data negli anni 1970 da John Backus nel suo famoso articolo intitolato "Can Programming Be Liberated from the Von Neumann Style? A Functional Style and Its Algebra of Programs". Backus, sviluppatore del Fortran e tra i principali contributori di Algol, non solo esaltò i pregi della programmazione funzionale, ma soprattutto evidenziò le limitazioni della programmazione imperativa, ancora troppo vicina all'architettura della macchina fisica. Rispetto al λ -calcolo, osservò come la sua notevole espressività conduce a una molteplicità di combinatori invece di concentrarsi su quei pochi che risultano adeguati nella maggior parte dei casi. Ispirandosi al linguaggio APL (A Programming Language) sviluppato negli anni 1960 da Kenneth Iverson per la programmazione algebrica su array, Backus propose FP (Functional Programming) come linguaggio algebrico basato su un piccolo insieme di combinatori dai nomi brevi che rendono i programmi particolarmente compatti.
- Negli anni 1970 Robin Milner sviluppò ML (MetaLanguage), nato originariamente come linguaggio di comandi per interagire con il dimostratore automatico di teoremi LCF. Esso include funzioni di ordine superiore, strutturazione a moduli e gestione delle eccezioni, nonché il pattern matching per gli argomenti delle funzioni e la possibilità di definire tipi di dati, come nel linguaggio Hope sviluppato in quegli stessi anni da Rodney Burstall. L'aspetto più significativo di ML è il suo ricco sistema di tipi, che supporta il polimorfismo per funzioni e strutture dati ed è in grado di inferire automaticamente il tipo di ogni espressione senza richiedere dichiarazioni esplicite di tipo. ML presenta anche aspetti imperativi come riferimenti a celle di memoria e relative istruzioni di assegnamento, oltre a istruzioni di ripetizione e di input/output.
- All'inizio degli anni 1980 David Turner sviluppò i linguaggi SASL (Saint Andrews Static Language), KRC (Kent Recursive Calculator) e Miranda in quest'ordine, dove ognuno costituisce un'evoluzione del precedente. Essi mettono a disposizione equazioni ricorsive come zucchero sintattico per il λ -calcolo, apposite espressioni per selezionare agevolmente elementi da liste, funzioni di ordine superiore basate sul currying e la valutazione pigra degli argomenti delle funzioni.

4.2 Haskell: Assemblaggio di Caratteristiche Funzionali

- Il linguaggio Haskell, così denominato in onore di Haskell Curry, venne progettato verso la fine degli anni 1980 da un comitato accademico europeo e americano per mettere a fattor comune gli aspetti più utili e innovativi derivanti dalla proliferazione di linguaggi funzionali più o meno puri durante quei decenni. La sua principale implementazione è il Glasgow Haskell Compiler di Peyton-Jones e Marlow.
- Diversamente dalla programmazione imperativa, in Haskell:
 - I programmi non sono istruzioni di assegnamento da eseguire nell'ordine stabilito dalle istruzioni di controllo del flusso, ma collezioni di funzioni da valutare.
 - Le esecuzioni dei programmi non sono sequenze di passi che modificano il contenuto della memoria, ma creano legami immutabili per le variabili e restituiscono i risultati della valutazione di funzioni.
 - Non ci sono effetti collaterali in quanto la valutazione di una funzione non può modificare variabili al di fuori del suo ambiente locale.
 - La conseguente trasparenza referenziale fa sì che venga restituito sempre lo stesso risultato ogni volta che una funzione viene valutata sugli stessi argomenti.
 - La ricorsione è il principale strumento linguistico per rappresentare l'iterazione e il tipo di dato ricorsivo lista è il principale tipo di dato strutturato disponibile, con la gestione dinamica della memoria per le liste completamente automatizzata.

- L'innalzamento del livello di astrazione verso l'uso di espressioni matematico-logiche al posto di sequenze di istruzioni è la caratteristica comune alle fasi dell'evoluzione della programmazione, che ha avuto come passaggi i linguaggi macchina e assemblativi, i primi linguaggi di programmazione imperativi, la programmazione strutturata e infine i linguaggi di programmazione dichiarativi come quelli funzionali. Ecco come viene programmato un ciclo nelle diverse fasi evolutive:

<code>/* assemblativo */</code>	<code>/* strutturato */</code>	<code>/* funzionale */</code>
<code> x := val_iniz;</code>	<code> x := val_iniz;</code>	<code> x = ciclo fun val_iniz 0 10</code>
<code> i := 0;</code>	<code> i := 0;</code>	<code> where</code>
<code>ciclo: x := fun(x, i);</code>	<code> while (i < 10)</code>	<code> ciclo f v i1 i2 = if (i1 >= i2)</code>
<code> i := i + 1;</code>	<code> x := fun(x, i);</code>	<code> then v</code>
<code> if (i < 10) goto ciclo;</code>	<code> i := i + 1;</code>	<code> else ciclo f (f v i1) (i1 + 1) i2</code>

- Queste sono le principali caratteristiche del linguaggio funzionale puro Haskell:
 - Sintassi equazionale: oltre che come λ -astrazioni, le funzioni possono essere espresse tramite una notazione algebrica che riduce al minimo l'uso delle parentesi e supporta in maniera naturale la ricorsione senza dover utilizzare operatori di punto fisso.
 - Stile dichiarativo: anziché mediante passi sequenziali, ogni funzione viene definita distinguendo tra molteplici casi sulla base di apposite guardie e posticipando la definizione di eventuali identificatori presenti nei vari casi ad apposite sezioni locali le cui linee sono indentate e senza terminatori.
 - Pattern matching: la possibilità di usare modelli per esprimere gli argomenti formali di una funzione semplifica i casi nella definizione della funzione, poi a tempo d'esecuzione gli argomenti effettivi vengono confrontati con tali modelli per determinare quale caso si applica.
 - Funzioni di ordine superiore: viene raggiunto un elevato livello di astrazione e di riuso del software grazie al fatto che una funzione da un lato può accettare funzioni come argomenti e dall'altro può restituire funzioni come risultati.
 - Valutazione pigra: gli argomenti effettivi vengono passati alle funzioni in base alla chiamata per necessità, quindi la loro valutazione viene ritardata il più possibile, i loro valori possono essere memorizzati per usi futuri ed è possibile gestire strutture dati illimitate.
 - Tipi e classi di tipi: oltre ai tipi scalari numerici, logici e carattere e ai tipi strutturati lista, stringa e tupla, sono presenti meccanismi per raccogliere i tipi in classi di tipi e costruire tipi enumerati, tipi unione e tipi ricorsivi assieme a eventuali operazioni su di essi.
 - Sistema di tipi forte ed espressivo: il sistema di tipi applica un numero significativo di controlli alle espressioni che compongono il programma, dando pertanto una certa garanzia di correttezza, ed è espressivo al punto da ammettere predicati sui tipi.
 - Inferenza di tipi: sebbene sia possibile accompagnare la definizione di ogni identificatore con la sua dichiarazione di tipo, quest'ultima non è obbligatoria in quanto il sistema di tipi è in grado di dedurre il tipo di qualunque espressione.
 - Polimorfismo di funzioni e strutture dati e overloading di operatori: una singola definizione per una funzione può trattare argomenti e risultati di tipi generici, così come una struttura può contenere dati di un tipo arbitrario e certi operatori sono applicabili a espressioni di tipi della stessa classe.
 - Gestione automatica della memoria: non ci sono puntatori né direttive esplicite per allocare o deallocare la memoria dinamicamente, in quanto la memoria viene automaticamente allocata quando serve e poi rilasciata quando non è più necessaria (garbage collection).
 - Sistema di input/output e strutturazione a moduli: queste due caratteristiche sono importanti affinché il linguaggio sia utilizzato in pratica, dove la prima è implementata in maniera funzionale mentre la seconda consente lo sviluppo di progetti software di grandi dimensioni.

- Esempio: Consideriamo il seguente programma Haskell per l'algoritmo di ordinamento quicksort:

```
quicksort :: (Ord a) => [a] -> [a]
quicksort [] = []
quicksort (x : xs) = quicksort xs_inf ++ [x] ++ quicksort xs_sup
  where
    xs_inf = filter (<= x) xs -- in alternativa: xs_inf = [x' | x' <- xs, x' <= x]
    xs_sup = filter (> x) xs  -- in alternativa: xs_sup = [x' | x' <- xs, x' > x]
```

Esso risulta essere estremamente conciso, se confrontato col corrispondente programma imperativo, e non necessita di puntatori per scorrere la lista che contiene gli elementi da ordinare, né presenta indicazioni per allocare la memoria necessaria per creare la versione ordinata della lista.

La prima linea dichiara `(::)` il tipo dell'identificatore `quicksort` stabilendo che quest'ultimo è una funzione `([a] -> [a])` che ha una lista come argomento (`[a]` a sinistra di `->`) e restituisce una lista come risultato (`[a]` a destra di `->`), dove `a` è una variabile di tipo. Gli elementi di queste liste sono di un generico tipo `a` su cui sono definiti operatori d'ordine (la premessa `(Ord a) =>` significa che `a` è un'istanza della classe di tipi `Ord`), quindi l'identificatore `quicksort` rappresenta una funzione soggetta a overloading, che è una forma ristretta di polimorfismo.

La seconda e la terza linea utilizzano il pattern matching per esprimere il caso base e il caso generale della definizione ricorsiva. Il primo caso stabilisce che se la lista argomento è vuota (`[]` a sinistra di `=`) allora anche la lista risultato è vuota (`[]` a destra di `=`). Il secondo caso stabilisce che se la lista argomento non è vuota, precisamente se ha un primo elemento indicato con l'identificatore `x` seguito (operatore infisso `:`) dalla lista eventualmente vuota dei successivi elementi indicata con l'identificatore `xs`, allora la lista risultato è ottenuta concatenando (operatore infisso `++`) il risultato dell'applicazione di `quicksort` a `xs_inf` con la lista `[x]` e poi con il risultato dell'applicazione di `quicksort` a `xs_sup`. L'applicazione di una funzione in notazione prefissa ha precedenza su tutti gli operatori infissi (da cui le parentesi tonde attorno a `x : xs`) ed è associativa da sinistra (come nel λ -calcolo).

La sezione `where` definisce gli identificatori `xs_inf` e `xs_sup` stabilendo che essi sono ottenuti da `xs` selezionando gli elementi opportuni tramite le specializzazioni `(<= x)` e `(> x)` degli operatori `<=` e `>`. La funzione `filter` è predefinita, ma se dovessimo definirla procederemmo nel seguente modo:

```
filter :: (a -> Bool) -> [a] -> [a]
filter _ [] = []
filter p (x : xs) | p x = x : filter p xs -- in alternativa: [x] ++ filter p xs
                  | otherwise = filter p xs
```

La funzione `filter` è polimorfa e, come dichiarato nella sua prima linea, ha come argomenti un predicato, cioè una funzione che restituisce un valore di verità (`a -> Bool`), e una lista (prima occorrenza di `[a]`), poi restituisce come risultato una lista (seconda occorrenza di `[a]`), pertanto è una funzione di ordine superiore. Nella dichiarazione del tipo di una funzione, il numero di argomenti della funzione è pari al numero di frecce che si trovano fuori da eventuali parentesi tonde.

Nella definizione ricorsiva viene usato `_` nel caso base perché il predicato non compare a destra di `=` e quindi non è necessario dargli un nome. Il caso generale comprende due sottocasi da valutare nell'ordine in cui sono scritti, ciascuno dei quali presenta una guardia tra `|` e `=`. Nel primo sottocaso si controlla se `p` è vero in `x` così da includere `x` nella lista risultato seguita (operatore infisso `:`) dall'applicazione ricorsiva di `filter` a `xs`, mentre nel secondo sottocaso (`otherwise`) l'elemento `x` non viene incluso.

Il tipo generico di `quicksort`, dichiarato per chiarezza anche se non obbligatorio dato che Haskell supporta l'inferenza di tipi, consente alla funzione, definita una sola volta, di essere applicata a liste di elementi di tipi specifici di volta in volta diversi come valori numerici, logici e caratteri:

```
quicksort [3, 2, 1]           [1,2,3]
quicksort [3, 2.5, -1]       [-1.0,2.5,3.0]
quicksort [True, False]     [False,True]
quicksort ['c', 'i', 'a', 'o'] "acio"
quicksort "ciao"            "acio"
```

4.3 Haskell: Espressioni, Tipi di Dati, Classi di Tipi

- Un'espressione in Haskell è formata da valori, operatori e identificatori, i quali determinano il tipo e il valore dell'espressione. L'espressione viene valutata, se e solo se è ben tipata, applicando gli operatori nel rispetto delle regole di precedenza e associatività (che è sempre da sinistra anche per quelli unari).
- I tipi di dati scalari predefiniti disponibili in Haskell sono i seguenti:
 - `Int` e `Integer` denotano l'insieme dei numeri interi, cioè i numeri espressi senza punto decimale, rappresentati in memoria in precisione fissa o arbitraria rispettivamente.
 - `Float` e `Double` denotano l'insieme dei numeri reali in precisione singola o doppia rispettivamente, espressi in virgola fissa o mobile (con la prima cifra della mantissa a sinistra del punto decimale).
 - `Bool` denota l'insieme dei valori di verità `False` e `True`.
 - `Char` denota l'insieme dei caratteri stampabili, di spaziatura e di controllo, espressi tra apici.
- Haskell introduce il concetto di classe di tipi come collezione di tipi correlati tra loro, nel senso che i loro insiemi di valori sono in qualche misura compatibili e condividono determinati operatori che quindi sono soggetti a overloading. Le classi formano una gerarchia dotata dei consueti meccanismi di ereditarietà: se S è una sottoclasse di C allora tutte le istanze di S sono istanze anche di C , così come tutti gli operatori disponibili in C sono disponibili anche in S .
- Questa è la gerarchia delle classi con i loro operatori principali per i tipi scalari summenzionati:
 - `Eq`:


```
(==)      :: a -> a -> Bool
(/=)      :: a -> a -> Bool
```
 - `Ord` sottoclasse di `Eq`:


```
(<)       :: a -> a -> Bool
(<=)      :: a -> a -> Bool
(>)       :: a -> a -> Bool
(>=)      :: a -> a -> Bool
min       :: a -> a -> a
max       :: a -> a -> a
```
 - `Bounded`:


```
minBound :: a
maxBound :: a
```
 - `Enum`:


```
succ      :: a -> a
pred      :: a -> a
```
 - `Num` sottoclasse di `Eq` (non di `Ord` perché i complessi non sono ordinabili) comprensiva degli interi:


```
(+)       :: a -> a -> a
(-)       :: a -> a -> a
(*)       :: a -> a -> a
negate    :: a -> a
abs       :: a -> a
signum    :: a -> a
```
 - `Integral` sottoclasse di `Num`, `Enum`, `Ord` (l'operatore `^` richiede che soltanto l'esponente sia intero):


```
div       :: a -> a -> a
mod       :: a -> a -> a
(^)       :: (Num b) => b -> a -> b
gcd       :: a -> a -> a
lcm       :: a -> a -> a
even      :: a -> Bool
odd       :: a -> Bool
```

- `Fractional` sottoclasse di `Num` comprensiva dei reali:

```
(/)      :: a -> a -> a
recip    :: a -> a
```

- `RealFrac` sottoclasse di `Fractional`:

```
truncate :: (Integral b) => a -> b
round    :: (Integral b) => a -> b
ceiling  :: (Integral b) => a -> b
floor    :: (Integral b) => a -> b
```

- `Floating` sottoclasse di `Fractional`:

```
pi       :: a
exp      :: a -> a
log      :: a -> a
(**)     :: a -> a -> a
logBase  :: a -> a -> a
sqrt     :: a -> a
sin      :: a -> a
cos      :: a -> a
tan      :: a -> a
```

dove:

- `Int` e `Integer` sono istanze di `Integral` e quindi anche di `Num`, `Enum`, `Ord`, `Eq`; inoltre `Int` è anche istanza di `Bounded`.
- `Float` e `Double` sono istanze di `Floating`, e quindi anche di `Fractional`, `Num`, `Eq`, oltre che di `Enum` e `Ord`.
- `Bool` e `Char` sono istanze di `Eq`, `Ord`, `Bounded`, `Enum`; inoltre `Bool` possiede i seguenti operatori specifici (quelli binari sono soggetti a cortocircuitazione per coerenza con la valutazione pigra):

```
(||)     :: Bool -> Bool -> Bool
(&&)     :: Bool -> Bool -> Bool
not      :: Bool -> Bool
```

ed è alla base dell'espressione condizionale `if c then e1 else e2` dove `c` deve essere di tipo `Bool` mentre `e1` ed `e2` devono essere dello stesso tipo (operatore ternario infisso cortocircuitato).

- Nelle espressioni gli operatori denotati da identificatori vengono scritti in notazione prefissa a meno che non siano binari e vengano racchiusi tra apici inversi, mentre gli operatori binari denotati da simboli vengono scritti in notazione infissa a meno che non vengano racchiusi tra parentesi tonde.
- L'applicazione di funzione, e quindi di operatore prefisso, ha precedenza sugli operatori binari infissi. I due operandi di un operatore binario possono essere di classi di tipi diverse a patto che una delle due classi sia sottoclasse dell'altra, nel qual caso è la sottoclasse a determinare il tipo dell'espressione.
- Esempi:

- `div 7 2` equivale a `7 'div' 2` e restituisce 3 perché `div :: (Integral a) => a -> a -> a`.
- `7 / 2` equivale a `(/) 7 2` e restituisce 3.5 perché `(/) :: (Fractional a) => a -> a -> a`.
- `4 + 1.0` produce 5.0 perché `4 :: Num` e `1.0 :: Fractional` con `Fractional` sottoclasse di `Num`.
- `9 :: Int` converte 9 dal tipo `Num` al tipo `Int` mentre `fromIntegral (9 :: Int)` lo riporta a `Num`.
- `minBound :: Int` e `maxBound :: Int` restituiscono il minimo e il massimo intero rappresentabile.
- 0.026 in virgola fissa è equivalente a 2.6e-2 in virgola mobile.
- `abs -8.3` viene interpretata come `abs` applicato a `-`, quindi va riscritta come `abs (-8.3)`.
- `False < True` è ben tipata, perché `Bool` è istanza di `Ord`, e restituisce `True`.
- `not not True` va riscritta come `not (not True)`, altrimenti prevale l'associatività da sinistra.

- Il programmatore può definire un nuovo tipo di dato scalare attraverso il meccanismo dell'enumerazione, specificando dopo `data` l'identificatore del nuovo tipo seguito da `=` e poi dagli identificatori dei costruttori dei valori del nuovo tipo separati da `|`. Ciascuno di questi identificatori, che devono essere tutti diversi tra loro, è una sequenza di lettere, cifre, sottotratti e apici che inizia con una maiuscola. La definizione del nuovo tipo può inoltre comprendere la clausola `deriving` per indicare le classi di tipi di cui il nuovo tipo è istanza (e di cui eredita pertanto gli operatori), nonché la definizione di operatori specifici del nuovo tipo. Un esempio di tipo di dato predefinito enumerato è `Bool`.

- Esempio di tipo di dato enumerato per i giorni della settimana:

```
data Giorno = Lunedì | Martedì | Mercoledì | Giovedì | Venerdì | Sabato | Domenica
    deriving (Eq, Ord, Enum)
domani :: Giorno -> Giorno
domani Domenica = Lunedì
domani g         = succ g
ieri :: Giorno -> Giorno
ieri Lunedì = Domenica
ieri g       = pred g
```

- I tipi di dati strutturati predefiniti disponibili in Haskell sono tupla, lista e stringa. Le tuple sono istanze di `Eq`, `Ord`, `Bounded` mentre le liste sono istanze di `Eq`, `Ord`. Le stringhe sono implementate come liste di `Char`.
- Una tupla è una sequenza di un prefissato numero $n \in \mathbb{N}_{\geq 2}$ di espressioni separate da virgole e racchiuse tra parentesi tonde. Queste espressioni possono essere di tipi diversi tra loro, quindi se l'espressione i ha tipo t_i allora la tupla ha tipo $(t_1, \dots, t_n)$. Le tuple possono annidarsi all'interno di altre tuple e sono utili ad esempio quando una funzione deve restituire più risultati. Due tuple sono confrontabili tramite gli operatori di `Eq` e `Ord` solo se sono dello stesso tipo, nel qual caso l'ordinamento considerato è quello lessicografico. Esistono inoltre degli operatori specifici per estrarre gli elementi delle coppie:

```
fst      :: (a, b) -> a
snd      :: (a, b) -> b
```

- Una lista è una sequenza di un arbitrario numero di espressioni tutte dello stesso tipo separate da virgole e racchiuse tra parentesi quadre, dove la lista vuota è rappresentata da `[]`. Se le espressioni sono di tipo t , allora la lista è di tipo `[t]`; in particolare, una stringa è di tipo `[Char]` e i caratteri che la costituiscono possono essere equivalentemente racchiusi tra virgolette senza separarli con virgole. Le liste possono annidarsi all'interno di altre liste e sono confrontabili tramite gli operatori di `Eq` e `Ord` anche se di lunghezze diverse a patto che siano dello stesso tipo, nel qual caso l'ordinamento considerato è quello lessicografico. Esistono inoltre degli operatori specifici per costruire liste:

```
(:)      :: a -> [a] -> [a]
(+++)    :: [a] -> [a] -> [a]
```

dove `:` è il costruttore canonico giacché $[v_1, \dots, v_n]$ si scompone unicamente in $v_1 : \dots : v_n : []$. Poiché i suoi due argomenti sono di tipi diversi, di fatto l'operatore `:` è associativo da destra.

- Il programmatore può ridenominare un tipo strutturato specificando dopo `type` il nuovo identificatore di tipo seguito da `=` e poi dal tipo da ridenominare. Ad esempio `String` è un alias di `[Char]` mentre `type Relazione a = [(a, a)]` crea un nuovo alias, facendo in entrambi i casi aumentare la leggibilità.
- È inoltre possibile definire i tipi unione e i tipi ricorsivi tramite il meccanismo dell'enumerazione esteso con la specifica di zero o più tipi dopo ogni costruttore. Ad esempio `data Numero = I Int | F Float` cui appartengono `I 9` ed `F 3.5`, oppure `data Nat = Z | S Nat` che dà luogo a `Z`, `S Z`, `S (S Z)`, ecc.
- In Haskell il valore di un'espressione può essere legato a una variabile mediante una definizione, il che è utile sia per incrementare la leggibilità che per non dover ricalcolare quel valore. L'identificatore di una variabile è una sequenza di lettere, cifre, sottotratti e apici che inizia con una minuscola o un sottotratto (in quest'ultimo caso il sottotratto deve essere seguito da almeno un altro carattere tra quelli ammessi); all'interno degli identificatori le lettere minuscole sono considerate diverse dalle corrispondenti lettere maiuscole (case sensitivity). Ad esempio `pi` è definita come `3.141592653589793` mentre `otherwise` è definita come `True`.

- La creazione del legame tra una variabile e il valore di un'espressione tramite `=` è di solito preceduta dalla dichiarazione del tipo della variabile tramite `::`, sebbene ciò non sia obbligatorio grazie al meccanismo di inferenza di tipi. Ogni variabile può comparire sulla sinistra di una sola definizione e il suo valore non può più cambiare (costante simbolica). L'ordine delle definizioni è irrilevante, quindi il programmatore deve solo preoccuparsi che non ci siano dipendenze circolari che farebbero divergere la valutazione.
- Le definizioni si dividono in globali e locali e possono essere arricchite con commenti. Questi devono essere racchiusi tra `{-` e `-}` oppure iniziare con `--` se residenti su un'unica linea.
- Esempio di definizioni globali equivalenti tra loro:

```
{- dall'alto verso il basso -}    {- dal basso verso l'alto -}

circonferenza :: Float           raggio :: Float
circonferenza = pi * diametro    raggio = sqrt 15.196

diametro :: Float               diametro :: Float
diametro = 2 * raggio           diametro = 2 * raggio

raggio :: Float                 circonferenza :: Float
raggio = sqrt 15.196            circonferenza = pi * diametro
```

- Una definizione locale consente di creare legami che hanno validità solo all'interno della definizione globale a cui sono associati. Essa si esprime attraverso il costrutto `where` o il costrutto `let-in`, ciascuno dei quali deve essere indentato e può contenere molteplici legami tra variabili e valori di espressioni a patto che tali legami stiano su linee diverse in quanto privi di terminatori.
- Esempio di definizioni globali con definizioni locali associate equivalenti tra loro:

```
{- dall'alto verso il basso -}    {- dal basso verso l'alto -}

circonferenza :: Float           circonferenza :: Float
circonferenza = pi * diametro    circonferenza = let
    where                         raggio = sqrt 15.196
        diametro = 2 * raggio     diametro = 2 * raggio
        raggio = sqrt 15.196     in
                                pi * diametro
```

■ftplf_8

4.4 Haskell: Funzioni, Guardie, Pattern Matching

- In Haskell il formato della definizione di una funzione è un'estensione del meccanismo per legare il valore di un'espressione a una variabile. Le due differenze più significative sono la presenza dell'operatore `→` nell'eventuale dichiarazione di tipo e la presenza degli argomenti formali nella definizione. Gli argomenti formali, che devono avere identificatori diversi tra di loro, non sono racchiusi tra parentesi tonde né sono separati da virgole. Il tipo dell' i -esimo argomento è quello compreso tra l' $i - 1$ -esima `→` e l' i -esima `→`, mentre il tipo del risultato è quello che segue l'ultima `→`, dove le frecce da considerare sono quelle che si trovano fuori da eventuali parentesi tonde e quindi gli argomenti sono tanti quanti queste frecce.
- La definizione di una funzione consiste in una o più equazioni (casi base e generale della ricorsione) ciascuna delle quali stabilisce che l'applicazione della funzione ai suoi argomenti, che si trova a sinistra di `=`, e il corpo della funzione, che si trova a destra di `=`, denotano lo stesso valore. In alternativa, può essere vista come una λ -astrazione, del resto l'applicazione della funzione ad argomenti effettivi segue la β -riduzione, cioè comporta la sostituzione degli argomenti effettivi ai corrispondenti argomenti formali all'interno del corpo della funzione.
- La definizione di una funzione può essere globale o locale e può stabilire comportamenti diversi a seconda del valore dei suoi argomenti. Questo può essere descritto ricorrendo a espressioni condizionali eventualmente annidate oppure, secondo uno stile più dichiarativo, dividendo la definizione in più casi, ciascuno dei quali è subordinato a una guardia, cioè un'espressione di tipo `Bool` scritta tra `|` e `=` che di solito è `otherwise` per l'ultimo caso, i quali vengono esaminati nell'ordine in cui sono elencati.

- Esempio: La funzione predefinita `signum :: (Num a, Ord a) => a -> a` potrebbe essere definita nel seguente modo attraverso espressioni condizionali annidate:

```
signum x = if (x > 0)
           then 1
           else
             if (x == 0)
               then 0
               else -1
```

ma in realtà è definita in uno stile più dichiarativo (e più leggibile) come segue:

```
signum x | x > 0  = 1
         | x == 0 = 0
         | x < 0  = -1
```

che è equivalente a:

```
signum x | x > 0      = 1
         | x == 0     = 0
         | otherwise = -1
```

L'argomento effettivo a cui è applicata va racchiuso tra parentesi tonde qualora sia un numero negativo oppure contenga operatori binari infissi, come in `signum (-1)` e `signum (2 * 5)`.

- Uno stile ancor più dichiarativo consiste nel definire le funzioni ricorrendo al pattern matching anziché alle guardie per la formalizzazione dei vari casi. Un pattern (modello) è un costrutto sintattico che specifica valori correlati e fornisce nomi per accedere a parti di tali valori, da usare nelle equazioni a livello di argomenti formali. In fase di applicazione della funzione considerata verrà verificato il matching (corrispondenza) degli argomenti effettivi ai pattern presenti a sinistra di `=`:
 - Ogni valore numerico, logico, carattere, enumerato, tupla, lista o stringa è un pattern che intercetta unicamente il valore stesso (utile per definire i casi base della ricorsione).
 - Ogni identificatore è un pattern che intercetta un qualsiasi valore, il quale viene legato all'identificatore così da potervi far riferimento a destra di `=` nella definizione del caso in esame.
 - Il simbolo `_` è un pattern che intercetta un qualsiasi valore al quale non si fa più riferimento a destra di `=` nella definizione del caso in esame.
 - Se p_1 e p_2 sono pattern, allora anche $p_1 : p_2$ è un pattern che intercetta una lista il cui primo elemento corrisponde a p_1 e la cui sottolista degli elementi successivi corrisponde a p_2 .
 - Se $p_1, \dots, p_n$ sono pattern, dove $n \in \mathbb{N}_{\geq 2}$, allora anche $(p_1, \dots, p_n)$ è un pattern che intercetta una tupla il cui i -esimo elemento corrisponde a p_i .
 - Se p è un pattern e x è un identificatore, allora anche $x@p$ è un pattern, detto ascrizione, che è utile nel caso in cui p non sia un valore scalare o un identificatore perché in quel caso a destra di `=` si può far riferimento tramite x all'intero valore intercettato senza dover ripetere p .
- Ogni pattern ha un tipo che è determinato dal tipo dei valori che corrispondono al pattern. Tutti i pattern per un argomento formale all'interno delle varie equazioni per una funzione devono essere dello stesso tipo e questo tipo deve coincidere con quello dell'argomento. Un identificatore può comparire al più una volta all'interno di un singolo pattern così come nella sequenza dei pattern per gli argomenti formali di una singola equazione (vincolo di linearità).
- I casi definiti tramite equazioni, pattern matching, guardie o una loro combinazione sono esaustivi quando coprono ogni possibilità rispetto ai tipi degli argomenti; se non sono esaustivi la funzione risulta essere parziale, cioè non definita per certi valori degli argomenti. Due casi si sovrappongono quando sono entrambi verificati per almeno una combinazione degli argomenti; in tale eventualità viene seguito il caso elencato per primo. I pattern `[]` e `_ : _` non si sovrappongono mai.

- Funzione per stabilire se una lista è una sottolista di un'altra lista (cioè compare all'interno):

```
sottolista :: (Eq a) => [a] -> [a] -> Bool
sottolista [] _ = True
sottolista (_ : _) [] = False
sottolista lx@(x : xs) ly@(y : ys) | prefisso lx ly = True
                                   | otherwise = sottolista lx ys
```

- Funzione per aggiungere un elemento all'inizio di una lista:

```
inserisci_elem :: a -> [a] -> [a]
inserisci_elem x xs = x : xs
```

- Funzione per aggiungere un elemento alla fine di una lista:

```
accoda_elem :: a -> [a] -> [a]
accoda_elem x xs = xs ++ [x]
```

- Funzione per concatenare due liste:

```
concatena :: [a] -> [a] -> [a]
concatena xs ys = xs ++ ys
```

- Funzione per invertire l'ordine degli elementi di una lista:

```
inverti :: [a] -> [a]
inverti [] = []
inverti (x : xs) = inverti xs ++ [x]
```

- Funzione per rimuovere tutte le occorrenze di un elemento da una lista:

```
rimuovi_elem :: (Eq a) => a -> [a] -> [a]
rimuovi_elem _ [] = []
rimuovi_elem x (y : ys) | x == y = rimuovi_elem x ys
                        | otherwise = y : rimuovi_elem x ys
```

- Funzione per fondere ordinatamente due liste ordinate:

```
fondi :: (Ord a) => [a] -> [a] -> [a]
fondi xs [] = xs
fondi [] ys = ys
fondi lx@(x : xs) ly@(y : ys) | x < y = x : fondi xs ly
                              | x == y = x : y : fondi xs ys
                              | x > y = y : fondi lx ys
```

- Esempi relativi agli algoritmi di ordinamento:

- Mergesort:

```
mergesort :: (Ord a) => [a] -> [a]
mergesort [] = []
mergesort [x] = [x]
mergesort lx@(x1 : x2 : xs) = fondi (mergesort xs1) (mergesort xs2)
  where
    (xs1, xs2) = dimezza lx

dimezza :: [a] -> ([a], [a])
dimezza [] = ([], [])
dimezza [x] = ([x], [])
dimezza (x1 : x2 : xs) = (x1 : xs1, x2 : xs2)
  where
    (xs1, xs2) = dimezza xs
```

- Quicksort: vedi fine Sez. 4.2.

4.5 Haskell: Funzioni Polimorfe, di Ordine Superiore, Anonime

- Il polimorfismo rappresenta la possibilità per una funzione di lavorare su argomenti di tipi di volta in volta diversi e per una struttura dati di contenere valori di tipi di volta in volta diversi. Ad esempio, grazie al supporto al polimorfismo disponibile in Haskell, la funzione identità può essere definita una volta per tutte come `identita x = x` favorendo pertanto il riuso di codice, invece di doverla ridefinire per ogni possibile diverso tipo del suo unico argomento formale `x`. Se il suo tipo non viene dichiarato, allora viene inferito il tipo più generale o principale, che è `identita :: t -> t`, cioè il tipo dal quale sono ricavabili tutte le istanze valide di tipi specifici per quella funzione.
- In aggiunta a tipi predefiniti, tipi definiti dal programmatore e costruttori di tipo come `->` per le funzioni, `()` per le tuple e `[]` per le liste, la dichiarazione del tipo di una funzione polimorfa contiene variabili di tipo, come ad esempio `a` in `identita :: a -> a`. Esse sono identificatori che rappresentano tipi generici, i quali verranno istanziati all'atto dell'applicazione della funzione ad argomenti effettivi. Ogni variabile di tipo è rigida all'interno del corpo della funzione, nel senso che tutte le occorrenze di argomenti formali di quel tipo dovranno poter essere istanziate allo stesso tipo specifico.

- Esempi:

```

- identita 1 :: (Num t) => t
- identita 2.5 :: (Fractional t) => t
- identita True :: Bool
- identita 'Z' :: Char
- identita ("ingresso", "uscita") :: ([Char], [Char])
- La variante f x = if x then x else x non può essere di tipo a -> a perché la prima occorrenza
  di x a destra di = deve essere di tipo Bool, quindi f non può essere polimorfa.
```

- L'overloading di una funzione può essere visto come una forma ristretta di polimorfismo. Il motivo è che nella dichiarazione di tipo di una funzione soggetta a overloading le variabili di tipo non denotano tipi completamente generici, ma istanze di una o più classi di tipi. Questo viene descritto facendo precedere l'espressione di tipo per la funzione dalle suddette classi, separate da virgole e racchiuse tra parentesi tonde, seguite da `=>`, come ad esempio in `(Num a, Ord a) => a -> a`.
- Poiché in Haskell non c'è differenza concettuale tra funzioni e valori, una funzione può essere passata come argomento, restituita come risultato o contenuta in una struttura dati. In particolare, una funzione che accetta funzioni come argomenti oppure restituisce funzioni come risultati è detta essere una funzione di ordine superiore. Il suo tipo contiene almeno un'occorrenza dell'operatore `→` racchiusa tra parentesi tonde.

- Esempi:

- Funzione predefinita `filter` che seleziona gli elementi di una lista in base a un certo predicato:

```

filter :: (a -> Bool) -> [a] -> [a]
filter _ [] = []
filter p (x : xs) | p x = x : filter p xs
                  | otherwise = filter p xs
```

- Funzione predefinita `map` che applica una certa funzione a tutti gli elementi di una lista:

```

map :: (a -> b) -> [a] -> [b]
map _ [] = []
map f (x : xs) = f x : map f xs
```

- Funzioni predefinite che riducono gli elementi di una lista a un singolo valore combinandoli a due a due da sinistra verso destra o viceversa tramite una certa funzione e un eventuale valore iniziale:

```

foldl :: (b -> a -> b) -> b -> [a] -> b    foldr :: (a -> b -> b) -> b -> [a] -> b
foldl _ i [] = i                          foldr _ i [] = i
foldl f i (x : xs) = foldl f (f i x) xs    foldr f i (x : xs) = f x (foldr f i xs)

foldl1 :: (a -> a -> a) -> [a] -> a         foldr1 :: (a -> a -> a) -> [a] -> a
foldl1 f (x : xs) = foldl f x xs           foldr1 _ [x] = x
                                           foldr1 f (x : xs) = f x (foldr1 f xs)

```

- Funzioni predefinite sulle liste basate sulle funzioni precedenti che combinano elementi:

```

sum :: (Num a) => [a] -> a                  product :: (Num a) => [a] -> a
sum = foldl (+) 0                          product = foldl (*) 1

minimum :: (Ord a) => [a] -> a              maximum :: (Ord a) => [a] -> a
minimum = foldl1 min                       maximum = foldl1 max

or :: [Bool] -> Bool                       and :: [Bool] -> Bool
or = foldr (||) False                     and = foldr (&&) True

reverse :: [a] -> [a]                     flip :: (a -> b -> c) -> b -> a -> c
reverse = foldl (flip (:)) []             flip f x y = f y x

```

e loro applicazioni:

```

sum [4, 5, 6] = foldl (+) 0 [4, 5, 6] = foldl (+) 4 [5, 6]
              = foldl (+) 9 [6] = foldl (+) 15 [] = 15

reverse [4, 5, 6] = foldl (flip (:)) [] [4, 5, 6] = foldl (flip (:)) [4] [5, 6]
                  = foldl (flip (:)) [5, 4] [6] = foldl (flip (:)) [6, 5, 4] []
                  = [6, 5, 4]

```

- Nelle liste capita spesso di dover ripetere la medesima computazione su ciascun elemento della struttura, trasformando tutti gli elementi allo stesso modo o filtrando ogni elemento in base alla stessa condizione. In Haskell ciò può essere anche espresso ricorrendo alle list comprehension, molto simili alla notazione insiemistica, con `[f x | x <- xs]` dove `f` è una funzione o `[x | x <- xs, p x]` dove `p` è un predicato. Inoltre è disponibile l'abbreviazione `[e_1 .. e_2]`, la quale genera la lista di tutti i valori compresi tra i valori di due espressioni di un tipo che è istanza di `Enum`.
- Esempi:

- Funzione per calcolare il successore di ogni elemento di una lista di valori di tipo enumerato:

```

lista_succ :: (Enum a) => [a] -> [a]      {- versione equivalente con map -}
lista_succ xs = [succ x | x <- xs]        lista_succ = map succ

```

- Funzione per filtrare gli elementi pari di una lista di interi:

```

lista_pari :: [Int] -> [Int]              {- versione equivalente con filter -}
lista_pari xs = [x | x <- xs, even x]     lista_pari = filter even

```

- Generazione della lista di coppie di cifre decimali di cui la seconda non è maggiore della prima:

```

coppie_cifre :: [(Int, Int)]
coppie_cifre = [(x, y) | x <- [0 .. 9], y <- [0 .. x]]

```

- In Haskell le funzioni possono essere definite anche al volo senza dare loro un nome seguendo quasi la stessa sintassi delle λ -astrazioni, dove λ è rimpiazzato da `\` mentre $.$ è rimpiazzato da `->`. Ad esempio `\x -> x + 1` è una funzione anonima che calcola il successore e la sua applicazione `(\x -> x + 1) 2` restituisce 3.
- Queste λ -espressioni possono essere usate pure all'interno delle definizioni di funzioni. L'effetto è quello di spostare gli argomenti formali dalla sinistra alla destra di `=` aprendo di conseguenza la via al currying, cioè alla possibilità di esprimere `f x_1 ... x_n = e` come `f = \x_1 -> ... \x_n -> e` e quindi di vedere alcune funzioni come specializzazioni di altre. Ad esempio `(==) 0` è il test di uguaglianza a 0 – manca il secondo argomento – mentre `(< 0)` è il test di negatività – manca il primo argomento.
- Esempio:

```
successore :: (Num a) => a -> a    {- lambda espressione -}    {- specializzazione -}
successore x = x + 1              successore = \x -> x + 1    successore = (+) 1
```

- Haskell fornisce l'operatore binario infisso `.` per la composizione di funzioni e l'operatore binario infisso `$` come applicazione di funzione su cui tutti gli altri operatori binari infissi hanno precedenza (diversamente dall'applicazione standard), i quali sono entrambi definiti tramite λ -espressioni:

```
(.) :: (b -> c) -> (a -> b) -> a -> c
f . g = \x -> f (g x)

($) :: (a -> b) -> a -> b
f $ x = f (x)
```

- Esempi:

```
- testa . coda è una funzione che restituisce il secondo elemento di una lista.
- signum 2 * 5 vale 5, signum (2 * 5) vale 1, signum $ 2 * 5 vale 1.
```

4.6 Haskell: Valutazione Pigrà, Input/Output, Moduli

- In Haskell gli argomenti effettivi di una funzione non vengono valutati all'atto della chiamata della funzione, ma nel momento in cui vengono usati dalla funzione invocata (chiamata per necessità e conseguente valutazione pigra). Abbiamo già visto che questo meccanismo dà naturalmente luogo alla cortocircuitazione degli operatori logici binari e dell'espressione condizionale. Esso consente inoltre la gestione di strutture dati illimitate interfogliando la loro generazione con la loro elaborazione.
- Esempio: La seguente funzione genera la lista di tutti gli interi compresi tra due interi dati:

```
genera_da_a :: Int -> Int -> [Int]
genera_da_a m n | m > n      = []
                | otherwise = m : genera_da_a (m + 1) n
```

mentre la seguente funzione genera la lista illimitata di tutti gli interi a partire da un intero dato:

```
genera_da :: Int -> [Int]
genera_da m = m : genera_da (m + 1)
```

La prima funzione può essere ridefinita nel seguente modo più compatto facendo uso della seconda:

```
genera_da_a' :: Int -> Int -> [Int]
genera_da_a' m n = take (n - m + 1) (genera_da m)
```

in quanto quest'ultima genera uno alla volta solo gli interi che sono man mano necessari all'esecuzione della funzione predefinita `take`, la quale restituisce la lista di un certo numero di elementi iniziali di una lista data.

- Mentre la valutazione pigra di espressioni genera valori seguendo le dipendenze tra i dati e senza creare effetti collaterali, le operazioni di input/output danno necessariamente luogo a effetti al di fuori della loro definizione. In Haskell sono rappresentate come azioni con effetto immediato che modificano in un certo ordine l'ambiente con cui un programma interagisce tramite l'acquisizione o la stampa di valori.
- Il costruttore di tipo `IO a` viene usato per denotare un'azione di input/output che produce un valore di tipo `a`. Esso si riduce a `IO ()`, dove `()` è il tipo unitario, quando il tipo dell'azione è irrilevante, altrimenti l'azione deve terminare con `return`. Le azioni di input/output non possono mischiarsi con le espressioni puramente funzionali e pertanto sono di solito raccolte all'inizio dei programmi Haskell.
- Le operazioni di input sono azioni che restituiscono i valori che sono stati acquisiti. Le operazioni di output non stampano direttamente valori, ma sono funzioni che producono azioni che stampano quei valori quando vengono eseguite. Queste sono le principali operazioni di input/output predefinite, dove l'acquisizione termina premendo invio nel caso di `getLine` o `<ctrl>d` nel caso di `getContents`:

```
getChar    :: IO Char           putChar    :: Char -> IO ()
getLine    :: IO String        putStr     :: String -> IO ()
getContents :: IO String        putStrLn  :: String -> IO ()
```

- I valori acquisiti possono essere collocati in variabili tramite `<-` all'interno del costrutto indentato `do`. Questo costrutto consente in generale di combinare in una singola azione più azioni di input/output, le quali vengono eseguite nell'ordine in cui sono scritte quando viene richiamata l'azione combinata, insieme a eventuali espressioni funzionali dove le variabili vengono legate ai relativi valori con `let` e `=`.
- Esempi:

- Azioni di input/output (le prime tre sono equivalenti, la prima e l'ultima sono combinate):


```
saluti :: IO ()           putStr "ciao\n" putStrLn "ciao" due_volte :: IO ()
saluti = do putChar 'c'   due_volte = do c <- getChar
          putChar 'i'     putChar '\n'
          putChar 'a'     putChar c
          putChar 'o'     putChar c
          putChar '\n'    putChar '\n'
```
- Programma per risolvere il problema delle torri di Hanoi (dischi, partenza, arrivo, intermedia):


```
main :: IO ()
main = do let ms = hanoi 10 'a' 'c' 'b'
          putStrLn $ show ms

hanoi :: Int -> Char -> Char -> Char -> [(Char, Char)]
hanoi 1 p a _ = [(p, a)]
hanoi n p a i | n > 1 = hanoi (n - 1) p i a ++ [(p, a)] ++ hanoi (n - 1) i a p
```

- Poiché le operazioni di input/output predefinite trattano solo caratteri e stringhe, la gerarchia delle classi di tipi comprende anche le classi `Read` e `Show` che sono in qualche modo collegate all'input/output. Ad esempio, è necessario aggiungere la derivazione da esse alla definizione del tipo enumerato `Giorno` altrimenti i valori di questo tipo non sono acquisibili né stampabili. Ne sono istanze `Int`, `Integer`, `Float`, `Double`, `Bool`, `Char`, i tipi definiti dal programmatore che sono esplicitamente dichiarati come derivazioni di `Read` e `Show` e le tuple e le liste di valori di tutti i tipi precedenti.
- Le classi di tipi `Read` e `Show` contengono operatori per convertire rispettivamente le stringhe acquisite in valori di determinati tipi oppure i valori di determinati tipi in stringhe da stampare. Queste sono le loro principali funzioni predefinite, dove la prima richiede di solito una conversione di tipo con `::` per il valore estratto dalla stringa acquisita:

```
read :: (Read a) => String -> a      show :: (Show a) => a -> String
```

- Esempio: Se la funzione `quicksort` (vedi fine Sez. 4.2) fa parte di un programma Haskell, la seguente azione combinata di input/output chiamata `main` deve essere aggiunta all'inizio del programma:

```
main :: IO ()
main = do putStrLn "Digitare la lista dei valori da ordinare:"
        s <- getLine
        putStrLn "Lista ordinata:"
```

la cui ultima azione è diversa a seconda del tipo dei valori da ordinare ed è una delle seguenti:

```
putStrLn $ show (quicksort (read s :: [Int]))
putStrLn $ show (quicksort (read s :: [Float]))
putStrLn $ show (quicksort (read s :: [Bool]))
putStrLn $ show (quicksort (read s :: [Char]))
putStrLn $ quicksort s -- stringa tra virgolette
putStrLn $ show (quicksort (read s :: [Giorno])) -- deriving (Read, Show)
```

- Un programma Haskell corposo viene solitamente organizzato in moduli che contengono definizioni di tipi, funzioni e variabili logicamente correlate tra loro. Tra questi moduli ce ne deve essere uno chiamato `Main` che esporta `main` di tipo `IO`, dove il valore di `main` è il valore dell'intero programma. Tutto ciò che è predefinito in Haskell viene messo a disposizione tramite il modulo `Prelude`.
- La definizione di un modulo comincia con `module` seguito dall'identificatore del modulo e poi da `where`. L'identificatore del modulo deve iniziare con una maiuscola e può essere accompagnato dall'elenco degli identificatori definiti nel modulo, separati da virgole e racchiusi tra parentesi tonde, che vengono esportati, altrimenti tutti gli identificatori globali definiti nel modulo sono considerati da esportare.
- Esempio di modulo per la gestione degli alberi binari:

```
module AlberoBin where

data AlberoBin a = Nil | Nodo a (AlberoBin a) (AlberoBin a)
  deriving (Read, Show)

cerca_albero_bin_ant :: (Eq a) => a -> AlberoBin a -> Bool
cerca_albero_bin_ant _ Nil = False
cerca_albero_bin_ant n (Nodo m sx dx) = n == m ||
  cerca_albero_bin_ant n sx ||
  cerca_albero_bin_ant n dx

cerca_albero_bin_post :: (Eq a) => a -> AlberoBin a -> Bool
cerca_albero_bin_post _ Nil = False
cerca_albero_bin_post n (Nodo m sx dx) = cerca_albero_bin_post n sx ||
  cerca_albero_bin_post n dx ||
  n == m

cerca_albero_bin_simm :: (Eq a) => a -> AlberoBin a -> Bool
cerca_albero_bin_simm _ Nil = False
cerca_albero_bin_simm n (Nodo m sx dx) = cerca_albero_bin_simm n sx ||
  n == m ||
  cerca_albero_bin_simm n dx

albero_bin_ric :: (Ord a) => AlberoBin a -> Bool
albero_bin_ric Nil = True
albero_bin_ric (Nodo n sx dx) = albero_bin_ric sx && minore_ug sx n &&
  albero_bin_ric dx && maggiore_ug dx n

where
  minore_ug Nil _ = True
  minore_ug (Nodo m sx dx) n = m <= n && minore_ug sx n && minore_ug dx n
  maggiore_ug Nil _ = True
  maggiore_ug (Nodo m sx dx) n = m >= n && maggiore_ug sx n && maggiore_ug dx n

cerca_albero_bin_ric :: (Ord a) => a -> AlberoBin a -> Bool
cerca_albero_bin_ric _ Nil = False
cerca_albero_bin_ric n (Nodo m sx dx) | n == m = True
  | n < m = cerca_albero_bin_ric n sx
  | n > m = cerca_albero_bin_ric n dx
```

- In un modulo è possibile far riferimento agli identificatori esportati da un altro modulo. Se quest'ultimo non è `Prelude`, allora va importato all'inizio scrivendo `import` seguito dal suo identificatore.
- Poiché un modulo può ridefinire un identificatore definito in `Prelude` o in un modulo importato, al fine di evitare ambiguità ogni occorrenza dell'identificatore deve essere concatenato tramite `.` all'identificatore del modulo in cui è definito. In alternativa, se l'identificatore è originariamente definito in un modulo importato e dopo `import` si scrive `qualified`, la concatenazione non serve per le occorrenze dell'identificatore ridefinito, però ogni occorrenza di ogni identificatore del modulo importato deve essere concatenato tramite `.` all'identificatore del modulo. A tale modulo si può dare un nome più breve scrivendolo dopo `as` alla fine dell'importazione qualificata.
- Esempio di modulo per la gestione dei grafi diretti:

```
module GrafoDir (GrafoDir, grafo_dir, adiac,
                 cerca_grafo_dir_amp, cerca_grafo_dir_prof) where

import qualified Lista as L

type GrafoDir a = ([a], [(a, a)])

grafo_dir :: (Eq a) => GrafoDir a -> Bool
grafo_dir (vs, as) = controlla_v vs && controlla_a as vs
  where
    controlla_v [] = True
    controlla_v (v : vs) = not (L.membro v vs) && controlla_v vs
    controlla_a [] _ = True
    controlla_a ((v1, v2) : as) vs = L.membro v1 vs && L.membro v2 vs &&
                                     not (L.membro (v1, v2) as) && controlla_a as vs

adiac :: (Eq a) => a -> [(a, a)] -> [a]
adiac _ [] = []
adiac v ((v1, v2) : as) | v == v1 = v2 : adiac v as
                        | v /= v1 = adiac v as

cerca_grafo_dir_amp :: (Eq a) => a -> GrafoDir a -> a -> Bool
cerca_grafo_dir_amp v (vs, as) i = L.membro i vs && cerca v [i] as [] 'a'

cerca_grafo_dir_prof :: (Eq a) => a -> GrafoDir a -> a -> Bool
cerca_grafo_dir_prof v (vs, as) i = L.membro i vs && cerca v [i] as [] 'p'

cerca :: (Eq a) => a -> [a] -> [(a, a)] -> [a] -> Char -> Bool
cerca _ [] _ _ = False
cerca v (u : us) as vis t | v == u = True
                        | v /= u = if (L.membro u vis)
                                then
                                  cerca v us as vis t
                                else
                                  if (t == 'a')
                                  then
                                    cerca v (us ++ adiac u as) as (u : vis) t
                                  else
                                    cerca v (adiac u as ++ us) as (u : vis) t
```

dove l'ultima funzione può essere definita in modo più dichiarativo come segue:

```
cerca :: (Eq a) => a -> [a] -> [(a, a)] -> [a] -> Char -> Bool
cerca _ [] _ _ _ = False
cerca v (u : us) as vis t | v == u = True
                           | v /= u = cerca' v (u : us) as vis t
  where
    cerca' v (u : us) as vis t | L.membro u vis = cerca v us as vis t
                                | otherwise      = cerca'' v u us as vis t
    cerca'' v u us as vis t | t == 'a' = cerca v (us ++ adiac u as) as (u : vis) t
                             | t == 'p' = cerca v (adiac u as ++ us) as (u : vis) t
```

- Oltre al modulo predefinito `Prelude`, il quale viene implicitamente importato in ogni altro modulo, Haskell mette a disposizione diverse librerie, ciascuna realizzata come un modulo da importare, tra cui citiamo in particolare:

- `Data.Array` e `Data.Ix` per gli array immutabili.
- `Data.Bits` per le operazioni sugli interi a livello di bit.
- `Data.Char` per le operazioni sui caratteri e sulle stringhe.
- `Data.Complex` per le operazioni sui numeri complessi in forma algebrica o trigonometrica.
- `Data.List` per le operazioni sulle liste.
- `System.IO` per le operazioni di input/output incluse quelle sui file. ■ftplf_10